B.TECH REVISION · PRIMER V1

# Tech Foundations Primer

The whole map in one read: how LLMs, APIs, RAG and agents actually work, and the B.Tech subjects that sit underneath them.

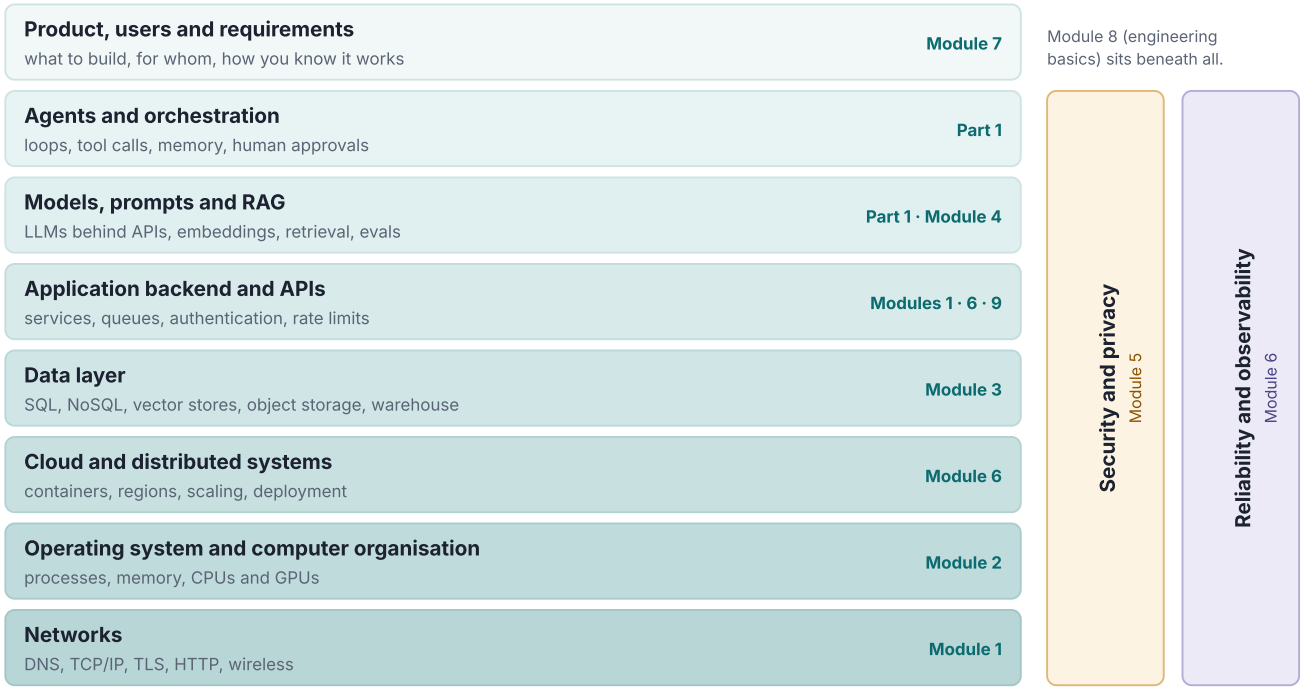

| | |
|---|---|
| **Part 0** | The big picture: one stack, nine modules, and how to read this |
| **Part 1** | LLMs and agentic AI: models, APIs, prompting, RAG, tools, agents, evals, safety, production |
| **Part 2** | The nine foundation modules, summarised: networks, OS, databases, ML, security, cloud, software design, engineering basics, programming |
| **Part 3** | One request through every layer: a worked end-to-end example |
| **Part 4** | Glossary, sources and what comes in v2 |


November 2026. Version 1 is the summary. Version 2 will take each subject in depth, one chapter at a time.

PART 0

# The big picture

Almost everything in modern AI products is ordinary software engineering wrapped around a model. If you understand the layers, the news stops sounding like magic.




Every AI feature you will meet is some combination of these layers. Security and reliability cut across all of them.

## How to read this document

- **Part 1** is the part to read first. It covers the vocabulary and mechanics of LLM products: APIs, prompts, RAG, tool use, agents, evaluation, safety and cost.
- **Part 2** summarises the nine foundation modules from the revision map. Each gets a page-sized overview: core ideas, why it matters for AI work, and three self-check questions.
- **Part 3** traces one request through every layer, so the pieces connect.
- **Depth target:** be able to explain each idea in a minute or two to a non-technical person, and reason about what breaks when it fails. This is not exam preparation.

**Marksheet to module map.** Networks: Computer Network Protocols, Data Communication, Network Programming. Systems: Digital Systems, Computer Organisation, OS. Data: Database Systems. ML: Data Mining, Predictive Analysis, Pattern Recognition. Security: Information Security. Cloud: Cloud Computing, Software Reliability. Design: Software Design Technology. Plus five first-year subjects and light programming basics.

**A note on freshness.** The concepts here are stable. Vendor details (model names, prices, exact API shapes, product features) change every few months, so treat any specific figure or name as something to check in current documentation.

PART 1

# LLMs and agentic AI

What you need to know to build with, buy, or judge systems that use large language models.

## 1.1 What an LLM actually is

- **A next-token predictor.** An LLM is a very large neural network trained to predict the next *token* (a chunk of text, roughly three-quarters of an English word) given everything before it. Answering, summarising, coding and apparent reasoning all emerge from doing that at huge scale.
- **Transformer, attention, embeddings.** The transformer is the architecture. *Attention* lets every token weigh every other token in the input when deciding what comes next. *Embeddings* turn tokens into vectors (lists of numbers) where similar meanings sit close together.
- **Training in stages.** *Pretraining* on a vast text corpus teaches language and general knowledge. *Post-training* (instruction tuning and preference-based methods) teaches the model to follow instructions and behave helpfully. *Reasoning models* get extra training to work through steps before answering.
- **Inference is what you call.** At inference time the weights are frozen and the model generates one token at a time. It does not learn from your conversation. Apps that seem to remember you are storing notes and re-feeding them.
- **Context window.** The model's working memory per request. Instructions, history, documents, tool results and the answer all count against it. Bigger windows cost more, run slower, and quality can degrade on very long inputs.
- **Sampling and temperature.** Generation is probabilistic. Higher temperature means more varied output. Even at low temperature, two identical calls can differ, so never assume exact repeatability.
- **Hallucination.** Fluent text that is not supported by fact. The model optimises for plausible continuations and has no built-in sense of what it does not know. Mitigations: ground answers in retrieved sources, require citations, verify with tools, and measure with evals.
- **Knowledge cutoff.** The model knows nothing after its training data ends. Anything recent must come from search or retrieval.
- **Size trade-off.** Larger models are more capable, slower and costlier. Use small, fast models for simple high-volume tasks and large ones for hard problems. *Hosted* models are easiest to use; *open-weight* models can be run yourself for control or privacy.

## 1.2 Calling a model: APIs

### FIRST, WHAT AN API IS

An API is a contract that lets one program ask another to do something. Web APIs run over HTTP: you send a request (method, URL, headers, usually a JSON body) and get back a response (status code, usually JSON).

| Concept | What to know |
|---|---|
| Methods | GET reads, POST creates or triggers, PUT/PATCH updates, DELETE removes. REST is the common style; GraphQL and gRPC are alternatives. |
| Status codes | 2xx success. 4xx means your request was wrong (401 not authenticated, 403 not allowed, 404 not found, 429 too many requests). 5xx means their side failed, so retrying may help. |
| Authentication | API keys or OAuth tokens in a header. Keys are secrets: keep them on the server, never in client code or a public repository. |
| Rate limits and quotas | Providers cap requests and tokens per minute. Expect 429s and design for them. |
| Idempotency | An operation that is safe to repeat. Retrying a non-idempotent call (charge a card, send an email) can do it twice, so use idempotency keys. |

| | |
|---|---|
| Pagination, webhooks, versioning | Large results arrive in pages. Webhooks mean the other service calls you when something happens. Versions let providers change without breaking you, until they retire an old one. |

#### WHAT IS SPECIFIC TO LLM APIS

- **Messages in, message out.** You send a list of turns (a *system* instruction, then *user* and *assistant* messages) plus settings such as model, maximum output tokens and temperature. You get back the generated message and a usage count of input and output tokens.
- **Stateless.** The API remembers nothing between calls. Your application resends whatever history it wants the model to see, which is why context management is your job.
- **Streaming.** Tokens can arrive as they are generated (typically server-sent events), so users see text immediately instead of waiting for the full answer.
- **Pricing is per token**, with output tokens costing more than input. *Prompt caching* makes a long, repeated prefix cheaper and faster. *Batch* endpoints trade speed for a lower price on bulk work.
- **Structured output.** You can ask for JSON that matches a schema. Always validate it in code anyway.
- **Failure handling.** Set timeouts, retry 429 and 5xx with exponential backoff and jitter, cap total retries, and log token usage and latency on every call.

```
POST https://api.example-llm.com/v1/messages
x-api-key: <SECRET>          content-type: application/json

{
  "model": "MODEL_NAME",
  "max_tokens": 500,
  "system": "You are an interview coach. Reply only with JSON.",
  "messages": [
    {"role": "user", "content": "Score this answer against the rubric: ..."}
  ]
}

-> { "content": [{"type": "text", "text": "{\"clarity\": 3, \"structure\": 4}"}],
     "usage": {"input_tokens": 412, "output_tokens": 96},
     "stop_reason": "end_turn" }
```

Illustrative shape only. Field names differ by provider; the ideas do not.

### 1.3 Prompting and context engineering

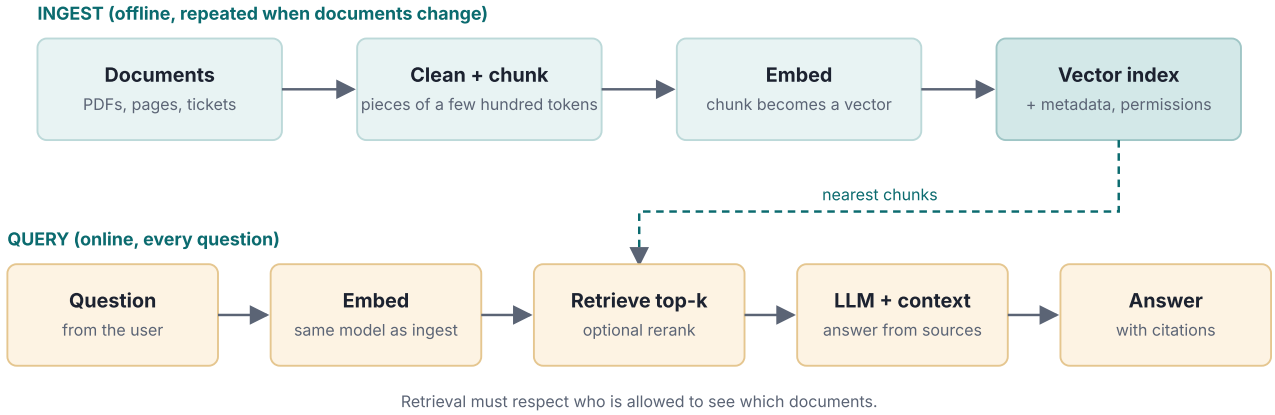
- **The prompt is everything the model sees:** system instructions, the task, examples, retrieved documents, tool descriptions and prior turns. The model has no other source of information about your situation.
- **Be specific.** State the goal, the audience, constraints, the exact output format, and what to do when unsure ("say you don't know rather than guess"). Show one or two examples of the output you want (*few-shot*). Separate instructions from data with clear delimiters or tags.
- **Thinking time.** Hard problems benefit from reasoning models or step-by-step instructions. Simple tasks do not, and it only adds cost and delay.
- **Context engineering** is the larger skill: deciding what goes into the window on each turn (which documents, which summary of old history, which tools). Clutter and stale content lower quality as surely as missing information does.
- **Treat prompts like code.** Keep them in version control, test on a fixed set of examples, and change one thing at a time.

### 1.4 Embeddings, vector search and RAG

- **Embedding.** A model converts text (or an image or audio clip) into a vector of hundreds or thousands of numbers. Similar meaning produces nearby vectors. Closeness is measured with *cosine similarity* or dot product.
- **Vector index.** A store that finds the nearest vectors to a query quickly using approximate search (graph indexes such as HNSW are common). It can be a dedicated vector database or an extension of a normal database, such

as pgvector for Postgres.

- **RAG (retrieval-augmented generation).** Instead of expecting the model to know your data, fetch the relevant pieces at question time and place them in the prompt, then instruct the model to answer from them.




RAG has two halves. Most quality problems come from the ingest half (bad chunks) or the retrieve step (wrong chunks), not from the model.

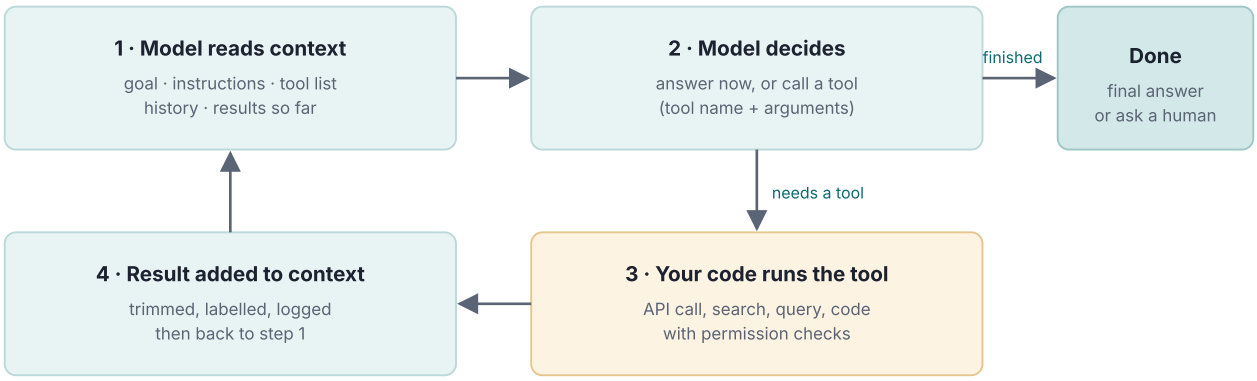
- **Chunking** is the most underrated step. Chunks that split a table or a thought in half retrieve badly. Use structure-aware splits with modest overlap, and store metadata (source, date, owner, permissions) with each chunk.
- **Hybrid search** combines keyword search (BM25) with vector similarity. Keywords win on names, codes and exact terms; vectors win on meaning. A *reranker* then re-scores the top candidates more carefully.
- **RAG versus the alternatives.** Fine-tuning changes a model's style and behaviour; it is an unreliable way to add facts. Stuffing everything into a long context is slow and expensive and quality can drop. RAG keeps data fresh, auditable and permissioned. Real systems often combine all three.
- **Evaluate the two halves separately.** Retrieval: did the right chunk appear in the top k? Generation: is the answer faithful to the retrieved text, and is it correct?
- **Failure modes:** wrong or missing chunks, stale data, the model ignoring the context, leaking documents a user should not see, and instructions hidden inside retrieved documents (see 1.8).
- **Agentic RAG.** Rather than one fixed retrieval, an agent decides what to search for, runs several queries and checks its own results.

## 1.5 Tool use (function calling)

- **The model cannot act; it can only write text.** Tool use bridges this. You describe each tool (name, purpose, argument schema). When useful, the model replies with a structured request such as `get_order(order_id=123)`. **Your code** runs it, checks permissions, and returns the result. The model then continues.
- **Tools can be anything:** web search, a database query, running code, reading a file, calling another API, sending an email.
- **Good tool design:** clear names and descriptions, narrow scope, helpful error messages, concise outputs, and the minimum permissions needed. The model reads your descriptions the way a new colleague reads documentation.
- **MCP (Model Context Protocol).** An open standard, introduced by Anthropic in late 2024, for connecting AI applications to tools and data through standard servers. Write a connector once and many AI apps can use it. A rough analogy is a universal port for tools.

## 1.6 Agents

An **agent** is a system where the model runs in a loop, choosing its own next step and using tools until the goal is met or a limit is reached.




The agent loop. The model never executes anything itself; your code is the hands, and your code decides what is permitted.

### WORKFLOWS VERSUS AGENTS

A *workflow* follows a path you coded (call the model, then another step, then another). An *agent* chooses its path at runtime. Start with the simplest design that works and add autonomy only where it earns its cost. Common patterns, as described in Anthropic's published guidance on building agents:

| Pattern | What it does |
|---|---|
| Prompt chaining | Break a task into fixed steps; each model call handles one and feeds the next. |
| Routing | Classify the input, then send it to a specialised prompt, tool or model. |
| Parallelisation | Run independent sub-tasks at the same time, or run the same task several times and compare. |
| Orchestrator and workers | A lead model splits the task dynamically and delegates to sub-agents with their own context. |
| Evaluator and optimiser | One call produces, another critiques, and the loop repeats until it passes a check. |

- **Memory.** Short-term memory is the context window. Long-term memory is information saved outside (notes, files, a database) and retrieved when relevant, which is RAG applied to the agent's own history.
- **Computer and browser use.** Agents can operate screens and websites. This is flexible but slower, costlier and more fragile than calling a proper API.
- **Reliability compounds.** If every step succeeds 95% of the time, a ten-step task succeeds only about 60% of the time (0.95 to the tenth power). Shorter chains, checks between steps and good tools matter more than clever prompts.
- **Cost compounds.** Every loop re-reads the growing context, so long runs get expensive quickly.
- **Autonomy ladder.** Suggest, then draft, then act with approval, then act and report, then act freely. Move up only as evals and trust justify it.
- **When not to use an agent:** the steps are known in advance (use a workflow or plain code), mistakes are costly or irreversible, latency matters, or you cannot define what success looks like.

## 1.7 Evaluation and observability

- **Evals are tests for AI behaviour.** Start with 20 to 50 realistic examples and the outcome you expect. Re-run them whenever the prompt, model, retrieval setup or tools change.
- **Graders:** code checks (schema, exact match, rules), human review, and *LLM-as-judge*, which is cheap and scalable but biased, so calibrate it against human ratings.
- **Trace every run:** prompts, retrieved chunks, tool calls and results, token counts, latency and cost. Without traces you cannot debug an agent.
- **Track** task success rate, groundedness, cost per task, latency, and how often a human has to step in.

## 1.8 Safety and security

- **Prompt injection.** Text the model reads (a web page, an email, a PDF, a tool result) can contain instructions that hijack its behaviour. There is no prompt that fully prevents this, so design the system so that a hijacked model cannot do serious harm.
- **The "lethal trifecta"** (a term from security researcher Simon Willison): an agent that has access to *private data*, is exposed to *untrusted content*, and can *send data out* can be tricked into leaking. Remove at least one of the three.
- **Controls that work:** least-privilege tools, read-only by default, sandboxed code execution, allow-lists for domains and actions, human approval for irreversible steps (payments, deletions, outbound messages), output validation, secrets kept out of prompts, and complete logs.
- **Privacy.** Send the minimum personal data, know the provider's retention and training terms, get consent, and redact where possible. Voice recordings of identifiable people are personal data. India's Digital Personal Data Protection Act, 2023 applies, so take proper legal advice before launching anything that stores them.
- **Judgement limits.** Models make confident mistakes and can reflect bias. Do not let one be the sole decision-maker on high-stakes outcomes.

## 1.9 Running it in production

| Concern | Levers |
|---|---|
| Cost | Tokens times calls. Reduce with smaller models for easy steps, shorter prompts, prompt caching, batching, capped loops, and routing hard cases up rather than sending everything to the largest model. |
| Latency | Stream output, run independent calls in parallel, use smaller models, cache, and keep the context lean. |
| Reliability | Timeouts, retries with backoff, idempotent actions, queues for slow work, fallbacks to another model or a cached answer, graceful handling of rate limits. |
| Change | Models get updated and retired. Pin versions, keep your evals, and re-run them before every upgrade. |
| Data | Logs and traces contain user content. Apply the same access control, retention and deletion rules to them as to the main database. |

## 1.10 Adjacent topics: fine-tuning, open models, multimodal, speech

- **Fine-tuning** trains a model further on your examples. It suits consistent style, format or a narrow task. It is not the right tool for teaching facts. Try prompting and RAG first.
- **Open-weight models** can be run on your own infrastructure for privacy, control or cost at scale, at the price of operating GPUs. Hosted APIs give faster access to the most capable models.
- **Multimodal models** accept images, audio, PDFs or video as well as text.
- **Speech pipelines** usually chain speech-to-text (ASR), then text analysis with an LLM, then optionally text-to-speech. Practical issues: latency for live conversation, accuracy across accents, speaker separation, word timestamps, and the fact that pace, pauses and filler words come from the audio signal, not from the transcript alone.

PART 2

# The nine foundation modules

A summary of each. Version 2 will take them one at a time, in depth.

## 1 Networks and the web

DEEP

Computer Network Protocols · Principles of Data Communication · Network Programming and Advanced Communication Networks · Wireless (conceptual)

- **Layers.** Application (HTTP, DNS), transport (TCP, UDP), network (IP), link (Ethernet, Wi-Fi). Each layer solves one problem and hides the rest.
- **URL to page.** DNS turns a name into an IP address, TCP opens a reliable connection, TLS encrypts it, then HTTP carries the request and response.
- **TCP versus UDP.** TCP guarantees ordered, reliable delivery with retransmission and congestion control. UDP is fast and unreliable, used for voice, video, games and DNS. QUIC (under HTTP/3) builds reliability on UDP.
- **Addressing.** IP addresses, subnets, NAT (many devices behind one public address), routing between networks.
- **HTTP.** Methods, status codes, headers, cookies, plus WebSockets and server-sent events for streaming.
- **Latency versus bandwidth.** Distance and round trips dominate small requests. CDNs and caching bring content closer to users.
- **Traffic management.** Load balancers, reverse proxies, API gateways, rate limiting.
- **Wireless and mobile.** Shared, variable links. Applications must tolerate drops and slow patches.

**Why it matters for AI work:** every model call is an HTTPS request, streaming is a long-lived connection, and timeouts, 429s and retries are networking facts. Voice products live or die on latency.

**Ready when you can explain:** what happens when you open a website; why retrying cannot fix a slow server; why mobile users on weak networks need chunked, resumable uploads.

## 2 Computer organisation and operating systems

MEDIUM

Digital Systems and Computer Organisation · Digital System Design Lab · Operating Systems · OS Lab

- **From gates to a CPU.** Logic gates build adders and memory; a CPU repeatedly fetches, decodes and executes instructions using registers, an ALU and a clock.
- **Memory hierarchy.** Registers, cache, RAM, SSD: each level is larger and far slower than the last. Programs that reuse nearby data run much faster (locality).
- **CPUs versus GPUs.** A few powerful cores versus thousands of simple ones. Neural networks are mostly matrix multiplication, which GPUs do in parallel.
- **What an OS does.** Shares hardware safely: processes, scheduling, virtual memory, file systems, drivers, system calls.
- **Processes and threads.** A process has its own memory; threads share one. Context switching lets many tasks appear to run at once.
- **Concurrency hazards.** Race conditions (two tasks change shared data in a bad order) and deadlocks. Async code is how servers wait on many network calls without blocking.
- **Containers.** Ordinary processes isolated by OS features (namespaces, cgroups) and sharing one kernel. Virtual machines emulate whole hardware.
- **Resource limits.** Out-of-memory, CPU throttling and full disks cause a large share of real outages.

**Why it matters for AI work:** GPU memory limits model size and context length, async I/O lets one server handle many LLM calls at once, and containers are how nearly everything is deployed.

**Ready when you can explain:** why GPUs suit LLMs; the difference between a process and a thread and what a race condition looks like; what a container really is.

## 3 Databases and data systems

DEEPEN

Database Systems · Database Systems Lab

- **Relational model.** Tables, keys, joins, SQL. Normalise to avoid duplication; denormalise deliberately for speed.
- **Indexes.** Usually B-trees. They turn full scans into quick lookups at the cost of slower writes and more storage.
- **Transactions and ACID.** Atomic, consistent, isolated, durable. Needed wherever money or orders are involved. Isolation levels trade safety for speed.
- **Query plans.** `EXPLAIN` shows what the database actually does, and is the first tool for slow queries.
- **Storage types.** Relational, document, key-value, wide-column, graph, time-series, vector, and object storage for files. Choose by access pattern.
- **Caching, replication, sharding.** Cache hot reads, replicate for failover and read scale, shard to spread data across machines. Each adds complexity and consistency trade-offs.
- **OLTP versus OLAP.** Operational databases serve the app; warehouses serve analytics. Pipelines move data from one to the other.

**Why it matters for AI work:** RAG is a database problem with embeddings, agent memory is storage, and logs, traces and evals are data. Any analytics product lives or dies on its data model.

**Ready when you can explain:** an index and its cost; when a document store beats SQL; what a vector database adds.

## 4 ML, data mining, predictive analysis and pattern recognition

DEEP

Data Mining and Predictive Analysis (and Lab) · Pattern Recognition · selected parts of Engineering Mathematics

- **Learning types.** Supervised (labelled examples), unsupervised (find structure), reinforcement (learn from reward), self-supervised (how LLMs are pretrained).
- **Workflow.** Define the target, collect and clean data, build features, train, validate, test, deploy, then monitor for drift.
- **Generalisation.** Train, validation and test splits; overfitting versus underfitting; the bias-variance trade-off; cross-validation.
- **Metrics.** Accuracy misleads when classes are imbalanced. Use precision, recall, F1, ROC-AUC; for numbers, MAE and RMSE.
- **Classic models.** Linear and logistic regression, decision trees, random forests and gradient boosting (strong on tabular data), k-means, k-nearest neighbours, SVMs, PCA.
- **Data mining.** Association rules (market basket), clustering, anomaly detection, text mining.
- **Neural networks.** Layers, activations, a loss function, backpropagation and gradient descent. Deep learning improves with data and compute.
- **Maths worth keeping.** Vectors and matrices, probability and Bayes' rule, basic statistics (variance, sampling, hypothesis tests), and derivatives as gradients.

**Why it matters for AI work:** everything in Part 1 rests on this. Classical models often beat LLMs on structured prediction, being cheaper, faster and explainable, and the discipline of evaluation transfers directly.

**Ready when you can explain:** why 99% accuracy can be useless; how you would detect overfitting; when to choose gradient boosting over an LLM.

## 5 Information security

DEEP

Information Security

- **Foundations.** The CIA triad (confidentiality, integrity, availability) and threat modelling: what are we protecting, from whom, through which entry points?
- **Authentication versus authorisation.** Who are you, versus what may you do. Passwords are stored as salted hashes, never plain. MFA, sessions, tokens (OAuth, JWT) and API keys as secrets.
- **Cryptography.** Symmetric encryption (fast, bulk data), asymmetric (key exchange, signatures), hashes (integrity). TLS combines them and certificates prove identity.
- **Web risks (OWASP Top 10).** Injection, broken access control, cross-site scripting, CSRF, misconfiguration, vulnerable dependencies, server-side request forgery.
- **Principles.** Least privilege, defence in depth, secure defaults, separate environments, secrets in a vault and never in code.
- **Network defences.** Firewalls, VPNs, rate limiting, DDoS protection.
- **Supply chain.** Dependencies, container images and AI-generated code all need review.
- **Privacy and compliance.** Encryption at rest and in transit, retention limits, consent, access logs, deletion on request.

**Why it matters for AI work:** an agent is a program that takes actions based on untrusted text, which is exactly the problem security exists for. Generated code often ships vulnerabilities, and recordings or personal data raise the stakes.

**Ready when you can explain:** how HTTPS protects a login; what SQL injection is and why parameterised queries fix it; how to spot the lethal trifecta in a design.

## 6 Cloud, distributed systems and reliability

DEEP

Cloud Computing · Software Reliability

- **Service models.** IaaS, PaaS, SaaS, and serverless (run functions, pay per use). Regions and availability zones.
- **Building blocks.** Compute (VMs, containers, functions), object storage, managed databases, queues and streams, load balancers, CDNs, identity and access management.
- **Containers and orchestration.** Package once, run anywhere, scale by adding replicas (Docker, Kubernetes).
- **CI/CD and infrastructure as code.** Automated tests and deploys, with environments defined in version-controlled files.
- **Distributed-systems truths.** Networks and machines fail, clocks disagree, partial failure is normal. CAP: during a network partition you must choose between consistency and availability. Eventual consistency is common.
- **Resilience patterns.** Timeouts, retries with backoff and jitter, circuit breakers, idempotency keys, queues to absorb bursts, horizontal scaling.
- **Reliability engineering.** Availability in "nines" (99.9% allows about 8.8 hours of downtime a year), SLIs and SLOs, error budgets, monitoring, logging, tracing, alerting, blameless postmortems.
- **Cost discipline.** Pay-as-you-go cuts both ways. Set budgets and alerts from day one.

**Why it matters for AI work:** an LLM app is a distributed system whose slowest and flakiest part is a remote API. Agent loops need retries, idempotency and timeouts. Scaling usually means queues and workers.

**Ready when you can explain:** idempotency and why retries need it; what 99.9% availability allows; when serverless fits and when it does not.

## 7 Software design, product and project management

MEDIUM-DEEP

Software Design Technology

- **Lifecycle.** Requirements, design, build, test, release, operate. Agile favours short cycles and feedback; both approaches fail without clear scope.
- **Requirements.** User stories, acceptance criteria and explicit non-goals. Prioritise with impact versus effort, MoSCoW or RICE.
- **Architecture.** Start with a monolith and split when you must. Use APIs as boundaries, aim for low coupling and high cohesion, and treat technical debt as a loan with interest.
- **Design principles.** Separation of concerns, DRY, YAGNI, SOLID. Learn to recognise common patterns when reading code.
- **Estimation.** Reliably unreliable. Break work into small pieces, time-box, surface risks early.
- **Testing and release.** Unit, integration, end-to-end tests; for AI features add evals. Feature flags, staged rollouts, monitoring, rollback.
- **Product thinking.** Problem before solution, MVP, activation and retention metrics, user research. With engineers, ask about trade-offs, risks and what extra time buys.

**Why it matters for AI work:** when AI writes much of the code, the leverage moves to deciding what to build, specifying it clearly and reviewing the result. AI features also need evals and product metrics to show whether they work.

**Ready when you can:** write a user story with acceptance criteria; explain monolith versus microservices trade-offs; define an MVP for a feature.

## 8 Engineering fundamentals

LIGHT · BREADTH

Engineering Chemistry · Basic Electrical Technology · Mechanics of Solids · Basic Electronics · Basic Mechanical Engineering

| Subject | Big ideas | Where it shows up |
|---|---|---|
| Engineering chemistry | Atomic structure and bonding, acids and bases, polymers, corrosion, materials and their properties. | Coatings and paints (resin binder, pigment, solvent, additives, curing); durability and weathering. |
| Basic electrical | Voltage, current, resistance (V = IR), power (P = VI), AC versus DC, transformers, three-phase supply, motors, safety. | Data centres and their power; why long-distance power uses high voltage (lower current, lower $I^2R$ losses). |
| Basic electronics | Diodes, transistors as switches and amplifiers, op-amps, sensors, ADC and DAC. | Transistors build logic gates, so this connects directly to Module 2. Microphones and sensors feed digital systems. |
| Mechanics of solids | Stress, strain, elastic modulus, tension, compression, shear, bending, factor of safety, fatigue. | Why structures, containers and plant equipment fail, and how safety margins are set. |
| Basic mechanical | Heat and energy, efficiency, fluids, gears and machines, manufacturing processes. | Production lines, pumps, mixing and packaging equipment. |

**Ready when you can explain:** Ohm's law and why high-voltage transmission loses less energy; stress versus strain; how a transistor can act as a switch.

## 9 Programming basics

LIGHT · READ AND REVIEW

Object Oriented Programming · Data Structures · Design and Analysis of Algorithms

- **OOP.** Classes and objects, encapsulation, inheritance, polymorphism, interfaces; prefer composition over deep inheritance.
- **Data structures.** Arrays, linked lists, hash maps (average constant-time lookup), stacks, queues, trees, heaps, graphs. Know what each costs.
- **Big-O.** How running time grows with input size: constant, logarithmic, linear, n log n, quadratic. Learn to spot an accidental quadratic loop.
- **Algorithm literacy.** Sorting, binary search, recursion, graph traversal (BFS and DFS), the idea of dynamic programming.
- **Reading and reviewing code.** Trace control flow and state, look for edge cases, error handling, security and hidden cost. This is the main skill when AI writes the first draft.
- **Version control.** Commits, branches, pull requests.

**Ready when you can explain:** why a hash map is fast; how to spot an $O(n^2)$ loop; what you would check when reviewing an AI-written function.

PART 3

# One request through every layer

A worked example to show how the modules connect. It is a hypothetical feature, used only for illustration: a student uploads a recorded mock-interview answer and gets scored feedback.

| # | What happens | Where it comes from |
|---|---|---|
| 1 | The app records audio and uploads it over HTTPS. On weak mobile networks the upload is chunked and resumable, so a dropped connection does not restart it. | Networks (1), Security: TLS (5) |
| 2 | The API server checks the user's token and consent, stores the audio file in object storage, and writes a record in the database. | Security (5), Databases (3), Cloud (6) |
| 3 | The server puts a job on a queue and returns immediately, so the user is not left waiting. A worker picks the job up. If the worker crashes, the job is retried, and the job is written so that running it twice does no harm. | Distributed systems and idempotency (6), OS: processes and async (2) |
| 4 | The worker runs speech-to-text to get a timestamped transcript, and computes audio measures such as pace, pauses and filler words. | ML and speech (4, Part 1.10) |
| 5 | The worker calls an LLM API with the rubric and the transcript, and asks for structured JSON scores with short justifications. It may retrieve a few example answers at different quality levels (RAG) so the scoring is anchored. | APIs, prompts, structured output, RAG (Part 1.2 to 1.4) |
| 6 | The JSON is validated against a schema. Odd or low-confidence results are flagged for review rather than shown as fact. Scores are saved. | Evals (Part 1.7), Databases (3) |
| 7 | The dashboard reads scores over time from an analytics store and shows trends. Notifications go out. | Databases: OLAP (3), Product (7) |
| 8 | Throughout: every step is logged and traced with cost and latency; alerts fire on failures; recordings are retained and deleted by policy; the eval set is re-run whenever the prompt or model changes. | Reliability (6), Security and privacy (5), Evals (Part 1.7) |

**What this shows.** Only steps 4 to 6 involve a model. Everything else is the engineering in the other modules. The model is one component, and most of the work that makes the product trustworthy sits around it.

**Where it tends to break:** uploads on bad networks (1), a slow or rate-limited model API (1.2, 6), scores that drift when the model is updated (1.9), and consent and retention for recordings (1.8). Each of these has an owner module.

## Sources and what comes in v2

Written from general knowledge of the field as of late 2026; vendor-specific details should be checked against current documentation. The agent patterns follow Anthropic's published "Building effective agents" guidance. The "lethal trifecta" framing is Simon Willison's. MCP was introduced by Anthropic and is now supported across many AI tools. Suggested learning resources for each module are listed in the revision map document.

**Version 2** will be one chapter per module: the full concept list, worked examples, diagrams, a hands-on exercise and a curated resource list. Suggested starting point: Module 1 (networks), since it is the weakest area and everything else builds on it.

PART 4

# Glossary

Short definitions for the terms you will keep meeting.

**Agent.** A model running in a loop with tools, choosing its own steps.

**API.** A contract for one program to request something from another.

**API key.** A secret string that identifies and authorises your application.

**ASR.** Automatic speech recognition: audio to text.

**Attention.** The mechanism letting each token weigh all others in the input.

**Batch API.** A cheaper, slower endpoint for bulk, non-urgent requests.

**BM25.** A classic keyword-ranking formula used in search.

**CDN.** Servers around the world that cache content close to users.

**Chunking.** Splitting documents into pieces for embedding and retrieval.

**Container.** An isolated process packaged with its dependencies.

**Context window.** The maximum tokens a model can consider in one request.

**Cosine similarity.** A measure of how closely two vectors point the same way.

**Embedding.** A vector representing the meaning of text, image or audio.

**Eval.** A repeatable test of an AI system's behaviour.

**Fine-tuning.** Further training of a model on your own examples.

**Function calling.** The model emitting a structured request to run a tool.

**GPU.** A processor with many simple cores, ideal for matrix maths.

**Hallucination.** Fluent output that is not grounded in fact.

**HNSW.** A graph-based index for fast approximate nearest-neighbour search.

**Idempotency.** A property of an operation that is safe to repeat.

**Inference.** Running a trained model to produce output.

**JWT.** A signed token carrying identity claims between systems.

**Kubernetes.** A system for running and scaling containers across machines.

**Latency.** The delay between a request and its response.

**LLM.** Large language model: a network trained to predict the next token.

**MCP.** Model Context Protocol, an open standard for connecting AI apps to tools and data.

**Multimodal.** Handling more than one type of input, such as text and images.

**OAuth.** A standard for letting one app act on your behalf without your password.

**Orchestrator.** The code or model that coordinates steps and sub-agents.

**Prompt.** All the input a model sees in a request.

**Prompt caching.** Reusing a repeated prompt prefix to cut cost and latency.

**Prompt injection.** Hidden instructions in content that hijack a model's behaviour.

**Rate limit.** A cap on how many requests or tokens you may use per period.

**RAG.** Retrieval-augmented generation: fetch relevant text, then generate from it.

**Reranker.** A model that re-scores retrieved candidates for relevance.

**REST.** A common style for web APIs built on HTTP methods and resources.

**SLO.** Service-level objective: a target for reliability or speed.

**SSE.** Server-sent events: one-way streaming from server to browser.

**Streaming.** Delivering output as it is generated.

**System prompt.** Standing instructions that frame the model's behaviour.

**Temperature.** A setting that controls randomness in generation.

**Token.** The unit of text a model reads and writes, about three-quarters of a word.

**Tool use.** Letting a model request actions that your code carries out.

**Transformer.** The neural-network architecture behind modern LLMs.

**Vector database.** A store optimised for finding nearest vectors.

**Webhook.** A call another service makes to your server when an event happens.

**Few-shot.** Including a few worked examples in the prompt.